Binary Search Trees (BST)

Deep Dive + Complexity + Real-World Systems

Java 8+

Session Structure

1. BST fundamentals
2. BST properties
3. Search, insert, delete
4. Complexity analysis
5. Balanced vs skewed trees
6. Real-world applications
7. Interview-level problems

What is a Binary Search Tree?

A BST is a Binary Tree with the property:

Left subtree < Root < Right subtree

For every node.

BST Property

For node X:

- All values in left subtree < X
- All values in right subtree > X

This enables:

Efficient search.

Why BST?

Arrays:

- Search O(n)
- Insert O(n)

BST:

- Search O(log n) (average)
- Insert O(log n) (average)

If balanced.

BST Node Structure

```
class TreeNode {
    int val;
    TreeNode left, right;

    TreeNode(int val) {
        this.val = val;
    }
}
```

Search in BST

Time: O(h)

Where h = height

```
static TreeNode search(TreeNode root, int key) {
    if (root == null || root.val == key)
        return root;

    if (key < root.val)
        return search(root.left, key);

    return search(root.right, key);
}
```

Average: O(log n)

Worst: O(n)

Insert in BST

Time: O(h)

```
static TreeNode insert(TreeNode root, int key) {
    if (root == null)
        return new TreeNode(key);

    if (key < root.val)
        root.left = insert(root.left, key);
    else if (key > root.val)
        root.right = insert(root.right, key);

    return root;
}
```

Delete in BST

Three cases:

1. Leaf node
2. One child
3. Two children

Time: O(h)

Delete Implementation

```
static TreeNode delete(TreeNode root, int key) {
    if (root == null) return null;

    if (key < root.val)
        root.left = delete(root.left, key);
    else if (key > root.val)
        root.right = delete(root.right, key);
    else {
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        TreeNode minNode = findMin(root.right);
        root.val = minNode.val;
        root.right = delete(root.right, minNode.val);
    }
    return root;
}

static TreeNode findMin(TreeNode root) {
    while (root.left != null)
        root = root.left;
    return root;
}
```


BST Complexity

Operation	Average	Worst
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

Worst case when tree is skewed.

Skewed vs Balanced BST

Balanced:

Height ≈ log n

Skewed:

Height ≈ n

Skewed behaves like linked list.

Inorder Traversal in BST

Inorder traversal gives:

Sorted order.

```
static void inorder(TreeNode root) {
    if (root == null) return;

    inorder(root.left);
    System.out.print(root.val + " ");
    inorder(root.right);
}
```

Time: O(n)

Validate BST

Time: O(n)

```
static boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

static boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;

    return validate(node.left, min, node.val) &&
           validate(node.right, node.val, max);
}
```

Lowest Common Ancestor (BST Optimized)

Time: O(h)

```
static TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null) return null;

    if (p.val < root.val && q.val < root.val)
        return lowestCommonAncestor(root.left, p, q);

    if (p.val > root.val && q.val > root.val)
        return lowestCommonAncestor(root.right, p, q);

    return root;
}
```


Kth Smallest Element

Uses inorder traversal.

Time: O(n)

```
static int kthSmallest(TreeNode root, int k) {
    java.util.Stack<TreeNode> stack = new java.util.Stack<>();
    TreeNode current = root;

    while (true) {
        while (current != null) {
            stack.push(current);
            current = current.left;
        }
        current = stack.pop();
        if (--k == 0) return current.val;
        current = current.right;
    }
}
```

Real World Use Case — Databases

Indexes use tree structures.

Fast search:
O(log n)

Used in:

- MySQL
- PostgreSQL
- File systems

Real World Use Case — Sorted Data Management

Examples:

- Leaderboards
- Ranking systems
- Priority ordering

Common Mistakes

- Ignoring skewed case
- Not handling duplicates
- Forgetting to update parent references
- Confusing BST with generic binary tree

Interview Discussion Points

- Why average O(log n)?
- Why worst O(n)?
- How to balance BST?
- Why inorder gives sorted order?

Practice Problems

Easy:

- Search in BST
- Insert node

Medium:

- Delete node
- Validate BST

Advanced:

- Recover BST
- Convert BST to sorted doubly linked list
- Build BST from preorder

Summary

BST provides:

- Ordered structure
- Efficient search (average O(log n))
- Foundation for AVL, Red-Black Trees
- Core to database indexing

Master:

- Search
- Insert
- Delete
- Validation
- LCA